LAPORAN PRAKTIKUM

TEKNOLOGI IOT(Modul7)

DOSEN:

Dr.Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

DISUSUN OLEH:

Nama : Adelia Rahmadani (25782030)
Kelas : TRI 3B

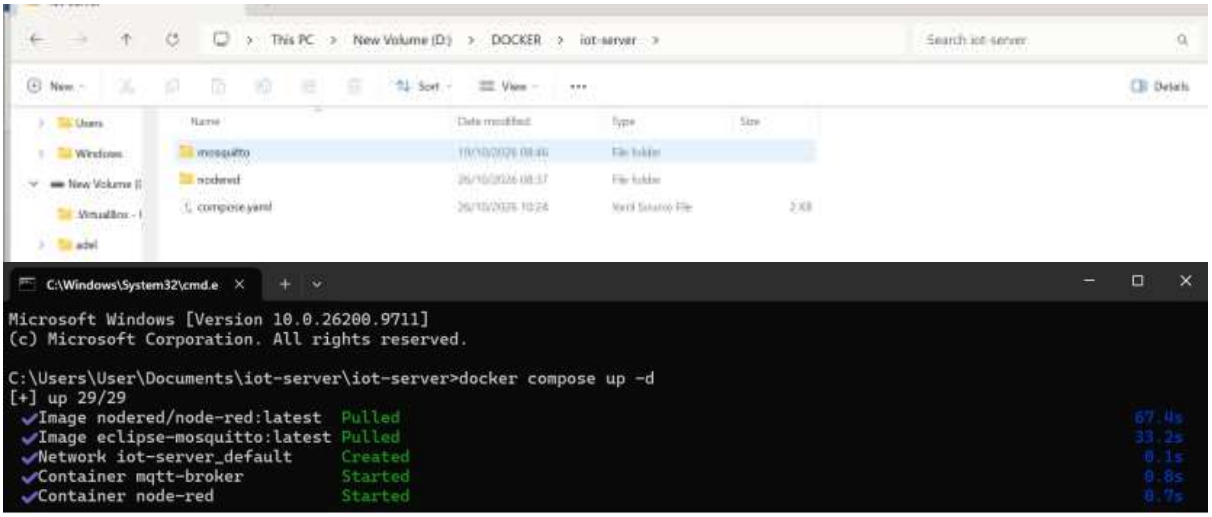
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

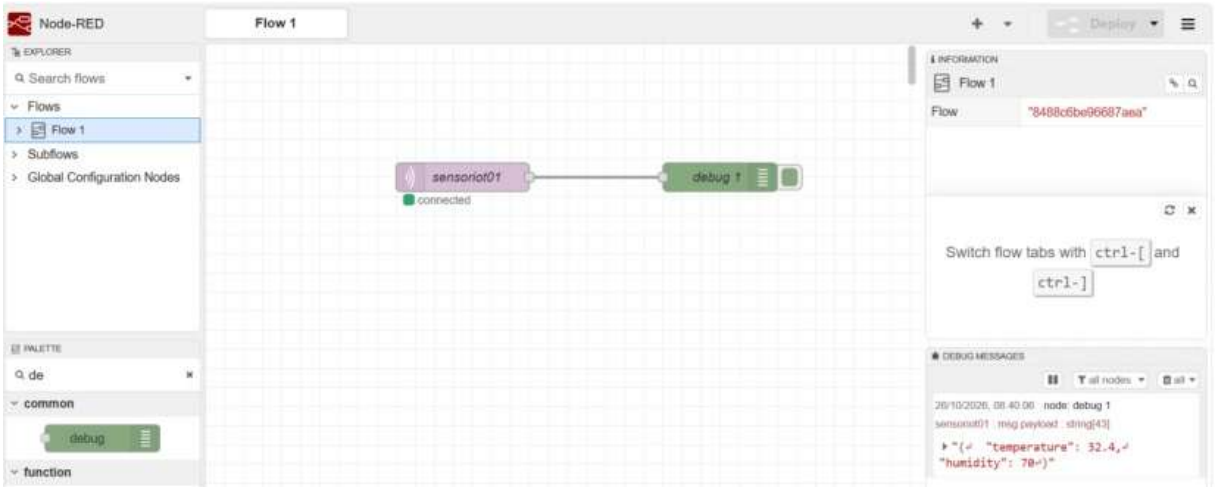
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

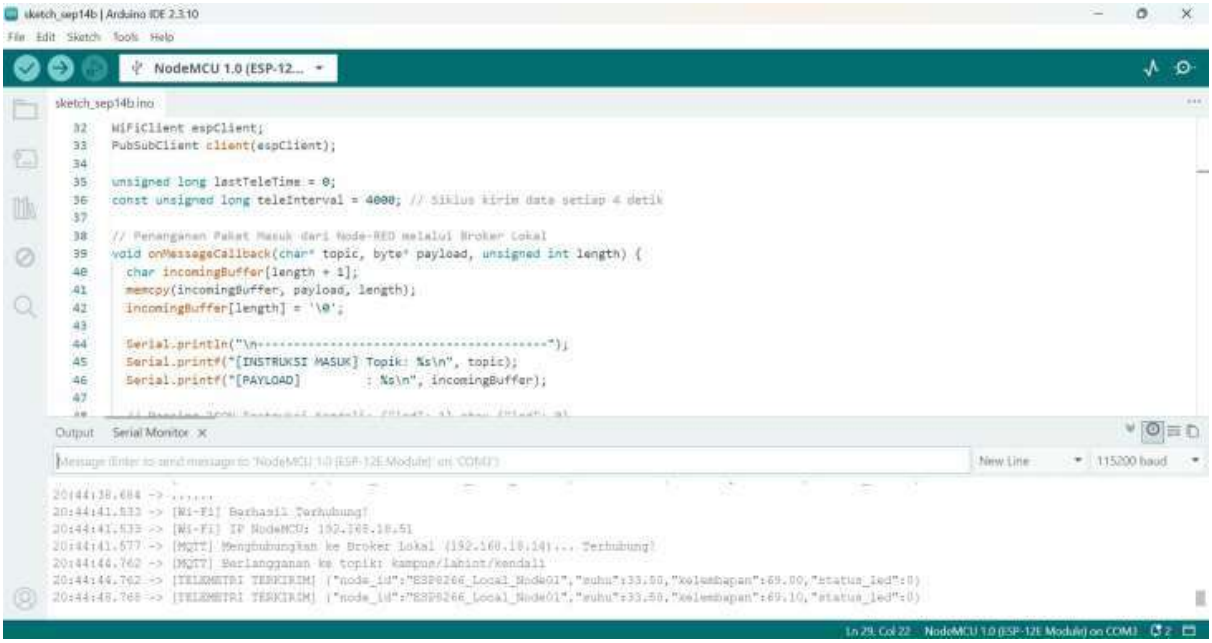
2026

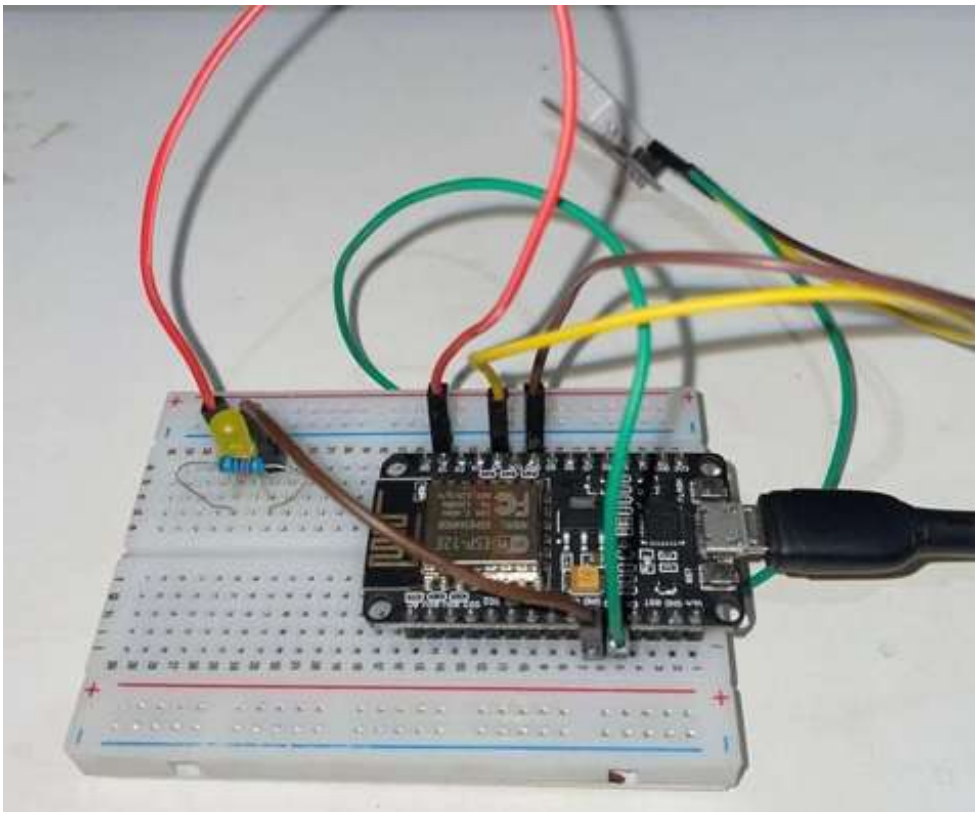
Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose



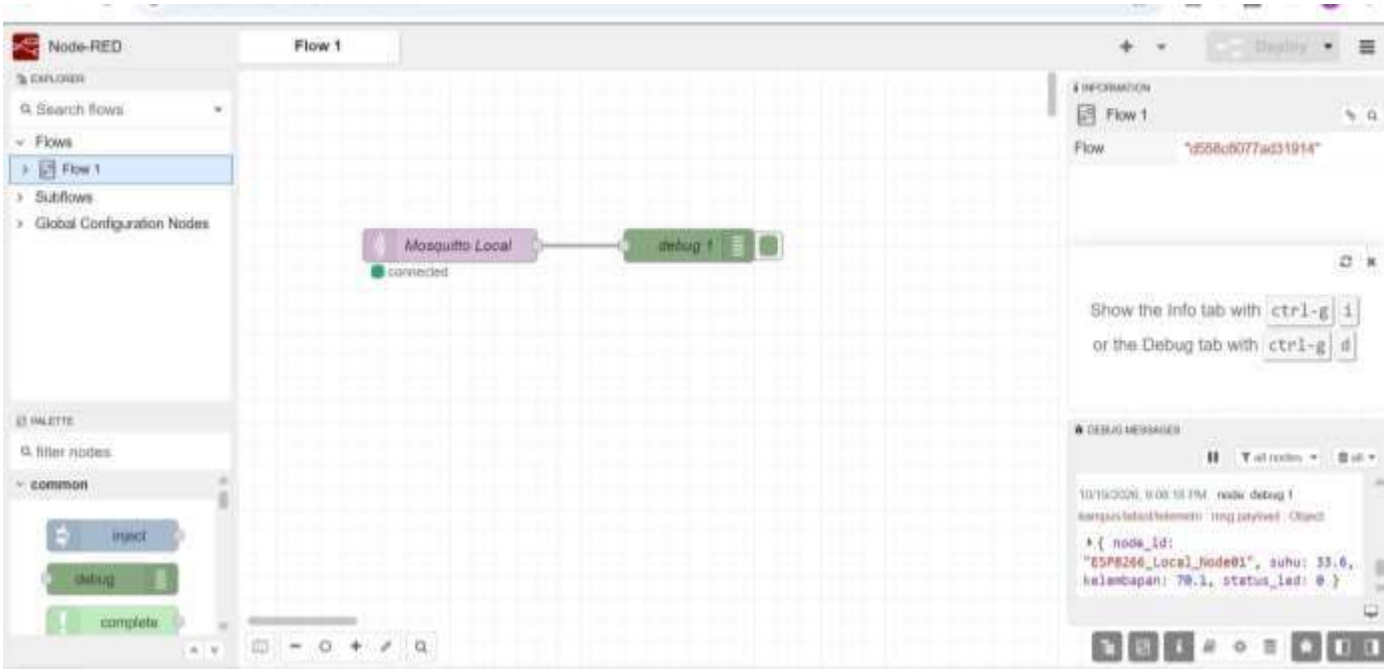


Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

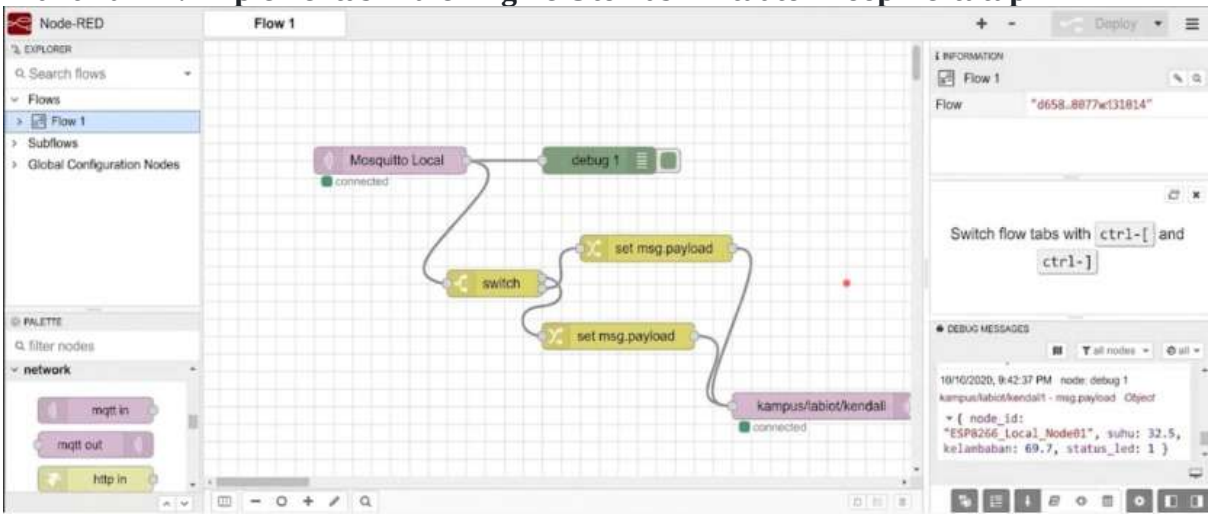


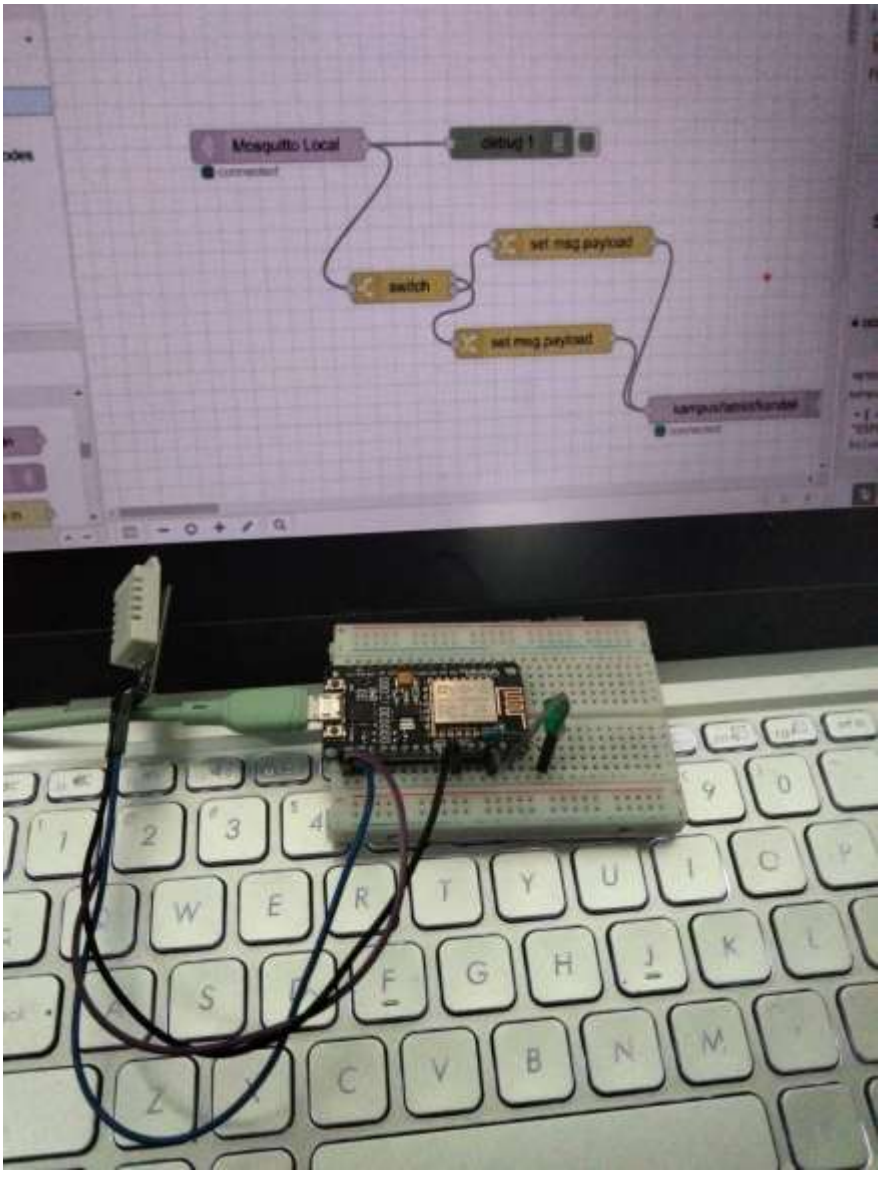


Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RE



Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup





Latihan 1: Analisis Jaringan Internal Docker & Service Discovery

1. **Faktor Penyebab Kegagalan Koneksi:** Penggunaan *localhost* (`127.0.0.1`) di dalam kontainer hanya mengarah ke antarmuka *loopback* milik kontainer Node-RED itu sendiri, bukan ke komputer *host* ataupun kontainer broker. Mengingat servis Mosquitto berjalan secara terisolasi pada kontainer terpisah (`mqtt-broker`), Node-RED secara otomatis menolak koneksi (*Connection Refused*) karena tidak ada servis MQTT aktif pada *port* 1883 di dalam kontainernya.
2. **Mekanisme Docker Embedded DNS & Bridge Network:** Ketika kontainer dijalankan melalui *User-Defined Bridge Network* di Docker Compose, Docker secara otomatis mengaktifkan server DNS internal (*Embedded DNS Server* di IP `127.0.0.11`). Server ini bertugas mengenali nama servis yang tercantum di berkas `docker-compose.yml` (seperti `mqtt`). Alhasil, saat Node-RED memanggil alamat `mqtt`, DNS internal Docker langsung menerjemahkannya ke alamat IP internal kontainer Mosquitto melalui mekanisme *service discovery*.

Latihan 2: Penanganan Masalah Konektivitas Jaringan & Subnet Firewall

1. **Alasan Gagalnya Koneksi NodeMCU:** Posisi NodeMCU (`192.168.43.102` dengan subnet `/24`) dan Laptop (`192.168.1.15` dengan subnet `/24`) berada di dua segmen jaringan yang berbeda. Tanpa bantuan perangkat *router* untuk menjembatani komunikasi antarsubnet (*inter-subnet routing*), paket data berupa *broadcast* maupun TCP/IP dari NodeMCU tidak akan mampu menembus batasan jaringan menuju laptop.
2. **Langkah-Langkah Teknis Penyelarasan Subnet:**
    - Hubungkan perangkat NodeMCU dan laptop ke jaringan Wi-Fi (*Access Point*) atau *hotspot* seluler yang sama.
    - Pastikan kedua perangkat berhasil mendapatkan alokasi IP otomatis (DHCP) pada segmen subnet yang serupa (misalnya sama-sama berada di rentang `192.168.1.x` atau `192.168.43.x`).
    - Perbarui nilai variabel `mqtt_server` pada kode sketsa Arduino IDE dengan alamat IP terbaru milik laptop yang sudah berada dalam satu subnet yang sama.

Latihan 3: Implementasi Filter Histeresis pada Rule Engine

Berikut adalah kode JavaScript yang digunakan pada *Function Node* Node-RED untuk meredam fluktuasi (*flickering*) pada status LED dengan menerapkan batas histeresis:

JavaScript
```
// Mengambil status LED sebelumnya dari memori konteks (bawaan: 0 / OFF)
let state = context.get('state') || 0;
let suhu = msg.payload.suhu;

// Logika pembatas histeresis dengan dua ambang batas
if (suhu > 31.5) {
    state = 1; // Lampu menyala hanya jika suhu melewati batas > 31.5°C
} else if (suhu < 30.0) {
    state = 0; // Lampu padam hanya jika suhu turun di bawah < 30.0°C
}

// Menyimpan kembali status mutakhir ke memori konteks
```

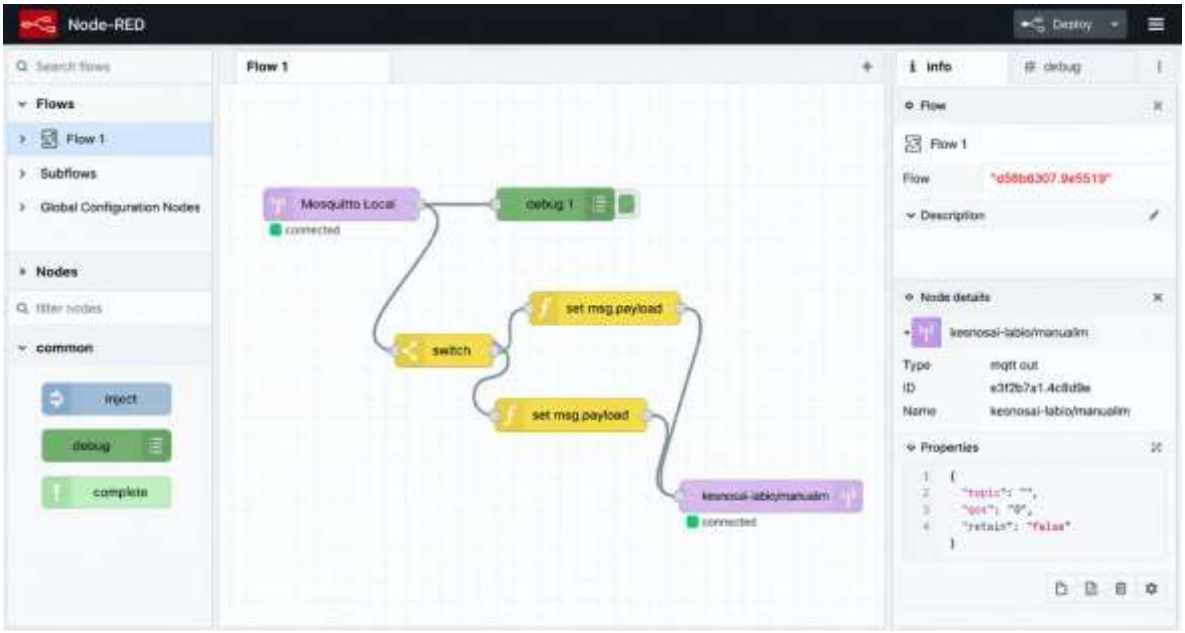
```
context.set('state', state);

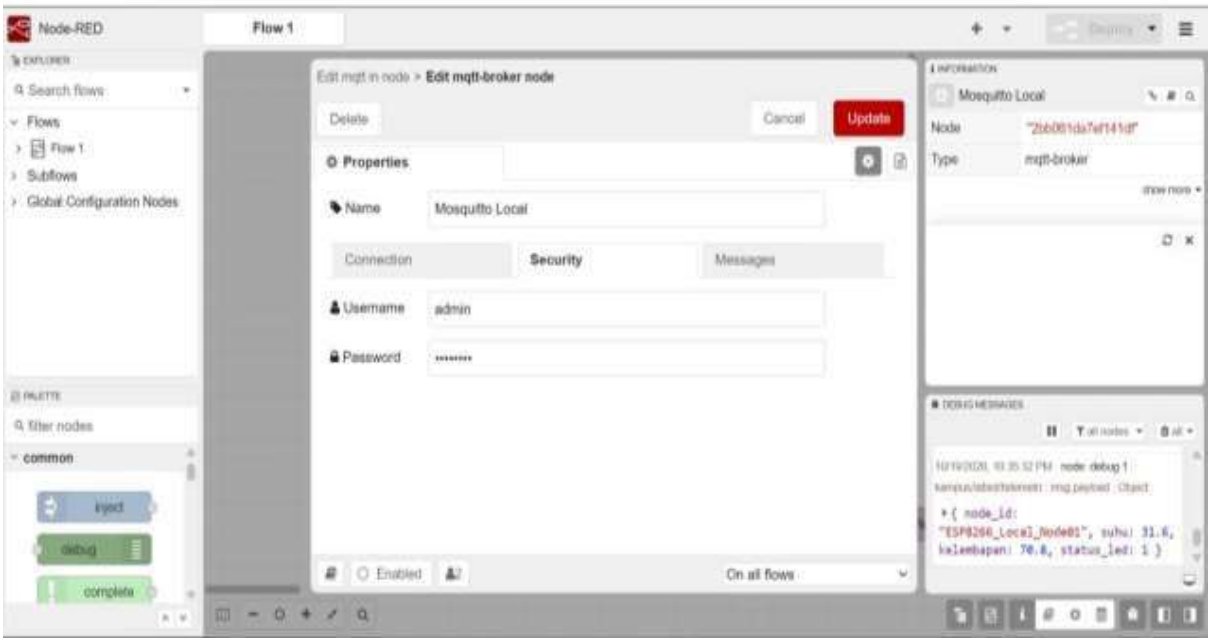
// Mengirimkan objek payload baru
msg.payload = { "led": state };
return msg;
```

Latihan 4: Analisis Mekanisme Volume Mount & Persistensi Data

1. **Konsekuensi Tanpa Volume Mount:** Karakteristik utama kontainer Docker bersifat sementara (*ephemeral*). Seandainya konfigurasi `volumes` dihilangkan lalu kontainer dimatikan lewat perintah `docker compose down`, seluruh rangkaian *flow* visual, pengaturan node, serta data kredensial di Node-RED akan lenyap secara permanen karena data tersebut hanya tersimpan di lapisan penulisan sementara (*writable layer*) milik kontainer.
2. **Perbedaan Mendasar Bind Mount vs Named Volume:**
    - **Bind Mount (`./nodered:/data`):** Berfungsi menghubungkan direktori atau berkas spesifik dari komputer *host* secara langsung ke dalam direktori kontainer. Jalurnya diatur secara manual oleh pengguna, sehingga berkas di dalamnya bisa disunting secara instan melalui *File Explorer* pada komputer *host*.
    - **Named Volume (`volume_name:/data`):** Pengelolaannya diserahkan sepenuhnya kepada sistem internal Docker di dalam direktori khusus (`/var/lib/docker/volumes/`). Lokasi ini terisolasi dari akses langsung pengguna dan diatur secara eksklusif menggunakan perintah-perintah Docker.

Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)





File Edit View

```
# ------------------------------------------------------------------------------
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ------------------------------------------------------------------------------

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

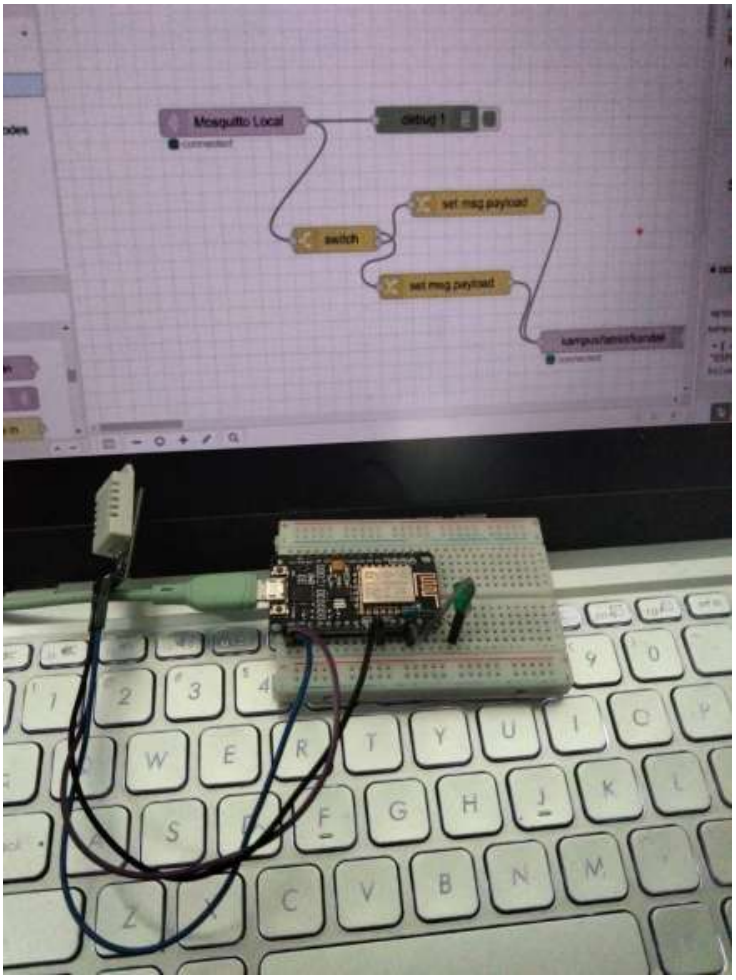
# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous false
password_file /mosquitto/config/password.txt

# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data

# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

\



Pada pelaksanaan praktikum 7.7, pengamanan pada server IoT dilakukan dengan menyematkan fitur autentikasi berbasis *username* dan *password* pada MQTT Broker Mosquitto. Konfigurasi `allow_anonymous false` diterapkan untuk memblokir setiap koneksi tanpa identitas, sementara `password_file /mosquitto/config/password.txt` berfungsi menentukan direktori penyimpanan data sandi pengguna dalam bentuk *hash*.

Sempat ditemui kendala saat konfigurasi di mana Mosquitto gagal membaca dokumen sandi akibat masalah hak akses (*permission*). Namun, kendala tersebut berhasil diatasi dengan menyesuaikan izin berkas sehingga broker dapat beroperasi kembali secara normal.

Berikutnya, Node-RED diatur menggunakan *username* admin beserta sandi yang valid agar dapat terhubung ulang ke broker. Perangkat ESP8266 juga wajib menyertakan kredensial serupa melalui fungsi `client.connect()`, sehingga mikrokontroler mampu menyambungkan koneksi MQTT, mengirimkan data sensor, merespons instruksi, serta mengontrol LED. Berkat penerapan sistem keamanan ini, akses tanpa izin (*anonim*) berhasil ditutup, menjadikan arus komunikasi antara ESP8266, Mosquitto, dan Node-RED jauh lebih terlindungi.